1. Why we chose File Handling and how it is relevant to our project

A trivia game based on Who Wants to Be a Millionaire needs a lot of good questions to keep players engaged. For our project, we put together 663 questions across three types: Multiple Choice, True or False, and Typed Answer. These questions are divided into Easy, Medium, and Hard difficulties, along with prize levels and visual graphics for the game show screens. Writing all 663 questions directly inside our Java code would have created thousands of lines of cluttered, repeated code. It would have made our project hard to read, increased loading times, and made simple changes annoying. If we found a typo or wanted to add a question, we would have had to change the main code and rebuild the whole application every time. We decided to use file handling so we could keep our game code separate from the question content. Storing questions in outside files made our project much cleaner and easier to manage.

2. What we learned through our research

Before starting on our code, we researched different ways to store and read data in Java. We compared several file formats, including JSON, CSV, and XML. CSV felt too simple to handle complex questions with lists of wrong choices or spelling variations. XML felt overly wordy with too many extra tags. We chose JSON because it is simple to read, handles lists and nested details well, and works easily with Java. We also looked into how Java opens files. We learned that using standard folder paths on a computer causes problems once the project is packaged into a runnable app, because bundled files exist inside the application package rather than as normal files on the hard drive. Using resource streams solved this problem by letting us open files directly from inside the app package. Finally, our research taught us how important it is to handle open files carefully so we do not waste computer memory. We learned to use automatic closing blocks in Java so files shut properly even if an error happens. We also made sure to use UTF-8 text encoding so special characters and accents in our trivia questions display correctly on any screen, and we used Google's Gson tool to turn JSON data into Java objects quickly.

3. How we applied File Handling in our project

We organized our files into clear folders and divided up the work based on our project roles. We grouped all our question files into nine separate JSON files based on question type and difficulty level, such as multiple_choice_hard.json. Our mentor Ivan suggested we create separate json files for each question, while this contributes to modularity and makes it easier to access, it also makes the project more complex and hard to deal with for backend integration which makes it not really fit for us. Other project settings and image graphics were placed into a shared resources folder so the application could easily find them. Aleksandar focused on building the backend file reader. He created a class that figures out which JSON file to open based on the requested difficulty and question type. His code opens the file safely using UTF-8 encoding and uses the Gson library to read the questions into Java objects. Once loaded, the questions get converted into actual question objects that power the main game. Efe worked on integrating file loading into the visual user interface. He set up the game screen to read the prize setup file and build the ladder display. He also loaded screen backgrounds and joker images into the visual panels. In addition, Efe

configured our project build settings so all the JSON files and graphics get packaged smoothly into the final application release.

4. What we learned from the implementation, including challenges, decisions, and its impact on our project

During development, we ran into a few practical issues that taught us important lessons about software design. One major issue was operating system differences: Aleksandar used Windows while Efe used macOS. Windows and Mac use different file path slashes, which caused file missing errors at first. We fixed this by switching to standard project resource paths with forward slashes, which work identically on all operating systems. We also ran into character encoding problems where special symbols in questions rendered as corrupted text on Windows computers. Explicitly setting UTF-8 encoding across all file readers solved this and guaranteed that text displays cleanly everywhere. Additionally, we added error checks so if a question file is missing or formatted wrong, the app stops immediately and shows a helpful error message instead of crashing silently during a game. To make the game extra reliable, we added backup options for settings files. If the prize level settings fail to load for any reason, the game falls back to a default set of prize values so the player can keep playing without interruption. Overall, adding file handling made our project modular, easy to update, and far more professional.